

모바일 엣지 컴퓨팅(MEC)을 위한 BiLSTM을 이용한 심층 강화 학습 기반 태스크 오프로딩 알고리즘

김 선 우, 이 창 우

국민대학교 소프트웨어학부

A Task Offloading Algorithm based on Deep Reinforcement Learning with BiLSTM for Mobile Edge Computing

Sunwoo Kim, Changwoo Lee

The School of Software, Kookmin University

● 투고분야 : 멀티모달 생성형 AI 현재와 미래

● 투고일 : 2024년 08월 23일

● Corresponding Author : Changwoo Lee

- Address : (02707) 77, Jeongneung-ro, Seongbuk-gu, Seoul, Republic of Korea
- TEL : +82-010-3260-7921
- FAX : +82-910-4868
- E-mail : leecw05@kookmin.ac.kr

모바일 엣지 컴퓨팅(MEC)을 위한 BiLSTM을 이용한 심층 강화 학습 기반 태스크 오프로딩 알고리즘

A Task Offloading Algorithm based on Deep Reinforcement Learning with BiLSTM for Mobile Edge Computing

Abstract

Task offloading in mobile edge computing(MEC) environments is an important research topic for enhancing user experience. This study proposes a novel approach to optimize task offloading in MEC based on deep reinforcement learning(DRL). This study replaces the LSTM(Long Short-Term Memory) in the existing QOCO(QoE-Oriented Computation Offloading) algorithm with BiLSTM(Bidirectional LSTM). The proposed method uses BiLSTM to improve the capture of dynamic workloads in edge nodes. This results in a performance enhancement of approximately 8% reduction in drop rate and a 6% reduction in the number of dropped tasks, as confirmed through experiments. These results show the advantages of combining DRL and BiLSTM in optimizing task offloading in MEC environments.

Keyword : Mobile Edge Computing(MEC), Task Offloading, Deep Reinforcement Learning, LSTM, BiLSTM

1. 서론

최근 모바일 애플리케이션이 매우 증가했으며, 비디오 스트리밍, 이미지 및 비디오 처리, 얼굴 인식, 온라인 게임 등 고도의 연산 작업을 하는 경우가 많아졌다. 하지만 모바일 디바이스(Mobile Device, MD)의 기술적 발전에도 불구하고, 전체적인 사용자 경험을 저하시키는 몇 가지 원인이 존재한다. 한정된 계산 능력과 배터리로 인해 작업 중단이나 처리 지연이 발생할 수 있다는 점이다[1]. 과거에는 MD의 부족한 컴퓨팅 성능을 보완하기 위하여, 모바일 클라우드 컴퓨팅(Mobile Cloud Computing, MCC)을 통해 이러한 계산 작업을 클라우드로 오프로드하여 진행했다.

그러나 MCC 도입에는 긴 전파 지연 시간 등의 단점이 있었으며, 이를 해결하기 위해 모바일 엣지 컴퓨팅(Mobile Edge Computing, MEC)이 등장했다. MEC는 MD 가까이에서 위치한 네트워크 엣지에서 클라우드 컴퓨팅 서비스를 제공한다. 이에 따라 낮은 지연 시간, 높은 대역폭, 실시간 계산 등을 가능하게 만든다. 한 예로, IoT 서비스에서 엣지 컴퓨팅을 활용하면, 기기의 발생 데이터를 브로커를 통해 바로 사용자에게 전송함으로써 IoT 서버의 부담을 줄일 수 있다[2]. 따라서

MD의 에너지 소비를 줄이고 지연 제약 조건을 만족시키기 위한 연구로, 작업 오프로드 결정 및 자원 할당 등의 오프로드 최적화가 진행되고 있다.

계산 오프로드 접근 방식은 전통적인 수치 최적화 접근 방식과 인공지능 기반 접근 방식으로 구분할 수 있다. 첫 번째는 일반적으로 사전에 설정된 매개변수가 필요하다. 또한 계산 복잡성으로 인해 최적해가 아닌 차선의 해를 도출하는 경우가 많다. 게다가 네트워크 환경이 바뀌면 솔루션을 업데이트하기 위해 동일한 절차를 다시 수행해야 한다는 단점이 존재한다[3]. 두 번째 방식은 인공지능(AI) 기반 접근 방식이다. 이 방식은 작업의 잠재적 지식을 학습하며, 낮은 복잡성으로 거의 실시간에 가깝게 최적 솔루션을 추론할 수 있다. 그러나 AI 모델 훈련 과정은 오프라인 학습을 위해 대량의 라벨링된 데이터가 필요하며, 온라인 추론 성능은 종종 불안정할 수 있다[4].

최근에는 다양한 심층 강화 학습(Deep Reinforcement Learning, DRL) 기반 알고리즘으로 네트워크의 동적 특성을 해결하고자 한다. DRL은 사전 지식 없이 환경 상태를 바탕으로 행동을 선택 및 탐색을 하고, 반복적인 상호 작용으로 장기적인 목표를 최적화할 수 있다. 이는 네트워크, 모바일 디바이스, 서버 간의 이질성에서 비롯된 문제를 효과적으로 해결하는 데 도움을 줄 수 있다[5]. 그러나 차원의 저주로 인해 이산적, 연속적, 조합적 행동 공간을 학습하는 데 어려움이 존재한다[6].

본 논문은 모바일 엣지 컴퓨팅의 오프로딩 및 관련 이론을 설명하고, 개선된 제안 방법, 마지막으로 실험 결과를 보여준다. 제안하는 방법은 QOCO(QoE-Oriented Computation Offloading Algorithm)의 알고리즘의 LSTM(Long Short Term Memory)을 BiLSTM(Bidirectional LSTM)으로 교체하여 성능 개선을 확인하였다. 그 결과 MD의 완료 태스크 개수는 평균 약 1%, 드랍율 및 드랍 태스크 개수는 약 8%, 6%의 성능 향상을 보였다.

2. 모바일 엣지 컴퓨팅의 오프로딩

모바일 에지 컴퓨팅(MEC) 분야는 사용자 경험 품질(QoE)을 향상시키기 위해 계산 오프로딩을 최적화하는 데 중점을 두고 다양한 연구가 진행되었다. MEC에서 계산 자원 관리, 작업 완료 시간 단축, 에너지 소비 최소화를 다루기 위한 여러 접근법이 제안되었다. 여기서는 특히 심층 강화 학습(DRL) 방법을 기반으로 하는 QOCO 알고리즘 연구를 검토한다.

2.1 MEC 시스템에서의 계산 오프로딩

최근 MEC 계산 오프로딩 연구는 AI 기반 솔루션으로 활발히 연구되고 있다. AI 접근법은 기계 학습과 같은 기술을 활용하여, 네트워크와 사용자 요구의 동적 변화를 실시간으로 반영하고, 최적화된 오프로딩 결정을 내릴 수 있다. 이러한 접근법은 특히 복잡한 계산 작업과 실시간 데이터 처리에 유리하며, 기존의 휴리스틱 접근법에 비해 높은 적응성과 효율성을 제공한다.

2.2 심층 강화 학습 접근법

최근에 DRL 기반의 계산 오프로딩 접근법이 주목받는다. DRL은 복잡한 환경에서 장기적인 보상을 최대화하는 정책을 학습할 수 있는 능력이 있어 MEC의 동적이고 분산된 특성에 적합하다[1]. 이 접근법은 계산 자원의 효율적인 활용과 사용자 QoE 향상을 동시에 달성할 수 있다.

2.3 제안된 QOCO 알고리즘

QOCO 알고리즘은 이러한 심층 강화 학습(DRL) 기반 접근법을 활용하여, 특히 개별 사용자의 QoE를 최대화하는 부분에 중점을 두고 진행한다. QOCO 알고리즘은 몇 가지 특징이 있다. 먼저 Dueling DQN과 LSTM을 사용하여 모바일 엣지에서의 동적인 워크로드를 효율적으로 처리한다. 그리고 각 모바일 디바이스(Mobile Device, MD)는 다른 MD의 정보 없이 독립적으로 오프로딩 결정을 내릴 수 있다. 또한 QoE를 최적화 하기 위해서 개별 사용자의 작업 완료율, 작업 지연, 에너지 소비를 종합적으로 고려한다.

QOCO 알고리즘은 해당 논문의 비교군들에 비해 작업 완료율은 14% 증가하였고 작업 지연 9% 감소하였다. 또한, 에너지 소비 측면에서는 6% 감소하였고 평균 QoE는 37% 이상 향상이라는 뛰어난 성능을 보였다. QOCO 알고리즘에서 주목할 점은 모바일 엣지에서의 동적인 워크로드에 대응하기 위하여, LSTM을 적용한 점이다[1]. 본 논문에선 이 부분을 중심으로 연구를 진행하였고 다른 시퀀스 모델인 BiLSTM을 적용하여 성능을 더욱 개선시켰다.

3. 학습 모델 이론

3.1 Dueling DQN

Dueling DQN은 강화 학습 DQN의 변형이다[7]. DQN은 Q-learning 알고리즘을 기반으로 하며, 각 액션의 가치를 예측하기 위해 하나의 네트워크를 사용한다. 반면에, Dueling DQN은 행동 가치를 예측하는 방법을 변형하는데, 바로 상태 가치와 장점을 분리하여서 예측한다. 상태 가치는 상태의 전체 가치를 예측하고, 장점은 각 액션이 발생시키는 가치를 나타낸다. 이를 통해 네트워크는 전체 상태의 가치와 각 액션의 상대적인 가치를 동시에 학습하여 보다 효율적으로 학습할 수 있다. (1)에서의 s 는 상태, a 는 행동을 나타낸다. 장점 가치 $B(s,a)$ 는 상태 가치 $C(s)$ 와 행동 가치 $Q(s,a)$ 의 차를 의미하나, 식별 불가(unidentifiable) 문제를 해결하기 (1)과 같이 정의된다.

$$Q(s,a) = C(s) + (B(s,a) - \frac{1}{|B|} \sum_a B(s,a')) \quad (1)$$

3.2 LSTM (Long Short Term Memory)

LSTM은 소실되는 기울기 문제를 효과적으로 해결할 수 있는 순환 신경망(Recurrent Neural Network, RNN)의 확장형 모델이다[8]. LSTM 모델은 입력의 장기 종속성을 유지하고 학습할 수 있도록 RNN의 메모리를 확장시켰다. 이러한 메모리 확장은 정보를 장기간 기억할 수 있다. 또한 메모리에서 정보를 읽고, 쓰고, 삭제하는 기능을 제공한다. LSTM 모델은 입력에서 중요한 특징을 포착하여 이 정보를 오랜 기간 동안 보존한다. 정보를 삭제할지 유지할지에 대한 결정은 학습 과정에서 정보에 할당된 가중치 값에 따라 이루어진다. 결과적으로, LSTM 모델은 어떤 정보를 유지하고 제거할지를 학습한다.

4. BiLSTM 기반 Task Offloading

4.1 BiLSTM (Bidirectional LSTM)

BiLSTM은 기존 LSTM 모델을 확장한 형태로, 입력 데이터에 두 개의 LSTM을 사용한다[9]. 첫 번째 단계 순방향 층에선 입력 시퀀스에 대해 LSTM이 적용되며, 두 번째 단계 역방향 층에선 입력 시퀀스의 역순 형태가 LSTM 모델에 입력된다. 이렇게 LSTM을 두 번 적용하면 장기 의존성을 학습하는 능력이 향상되어 모델의 정확도가 높아진다[10]. 또한 시계열 데이터 예측 문제에도 더 뛰어나다. 단방향 LSTM 모델은 한 방향, 과거에서 현재로만 훈련되지만, BiLSTM이 양방향 학습으로 데이터와 관련된 추가적인 특징을 포착할 수 있기 때문에 더 유리하다[11].

QOCO 알고리즘에선 미래의 워크로드를 예측하려고 Edge Node(EN)들의 워크로드 과거 데이터를 입력으로 받는 LSTM을 사용했다. 본 논문에선 BiLSTM이 시계열 데이터 예측에서 LSTM보다 더 유리하기 때문에, LSTM 대신 BiLSTM을 적용하여 QOCO 알고리즘 성능을 향상시켰다.

4.2 제안 및 아키텍처 설명

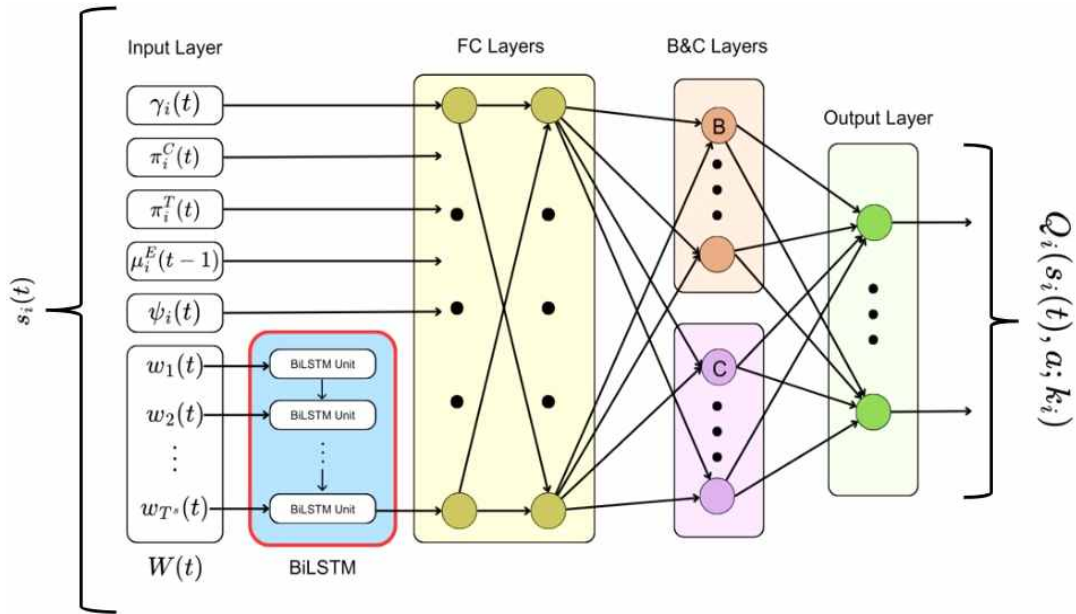


Fig. 1. Proposed model architecture for BiLSTM offloading

QOCO 모델은 각 타임슬롯 t 의 상태를 입력받아, FC-Layer와 B&C Layer를 거쳐 출력인 $Q_i(s_i(t), a_i, k_i)$ 구한다. Dueling DQN 기반으로 네트워크 B는 장점, C는 상태가치를 나타낸다. 그리고 탐험(exploration)인 경우를 제외하면, 각 타임슬롯 때 $Q_i(s_i(t), a_i, k_i)$ 을 최대화시키는 행동을 선택한다. 상태는 순서대로 태스크 크기 $\gamma_i(t)$ 계산, 전송 큐의 정보 $\pi_i^C(t)$, $\pi_i^T(t)$, 이전 타임슬롯 때의 모든 ENs에 있는 MD _{i} 의 큐의 길이 $\mu_i^E(t-1)$, MD _{i} 의 배터리 레벨 $\psi_i(t)$, 모든 ENs의 활성화 큐 개수를 나타내는 과거 데이터 행렬 $W(t)$ 이다. 행동은 MD가 어떤 EN에 오프로드할지 결정하는, 원-핫 인코딩 벡터이다.

QOCO 논문에서는 결과적으로 워크로드의 변동을 나타내는 $W(t)$ 을 LSTM에 입력으로 집어넣었다[1]. 결과적으로 LSTM은 워크로드의 변화를 학습하게 되고, 예상되는 워크로드 변화를 캡슐화된 하나의 값을 출력으로 내뱉는다. 이는 FC Layers에서 모델이 ENs의 워크로드 변화를 고려하여 행동을 선택한다. BiLSTM은 두 개의 독립적인 LSTM 층으로 구성되어 있다. 하나는 입력 시퀀스를 과거에서 현재로, 다른 하나는 입력 시퀀스를 현재에서 과거로 처리한다. 본 논문은 LSTM을 BiLSTM으로 변경하여, 모델이 보다 워크로드 변화를 잘 고려하여 성능 향상을 계획했다. 입력인 $W(t)$ 은 각 타임슬롯을 지나가는 시계열 데이터이다. LSTM과 달리 양방향으로 학습으로 시계열 예측에 더 유리하기에, 제안 모델이 기존 모델보다 더 좋은 성능을 보였다.

5. 실험 결과

BiLSTM 기반 태스크 오프로딩의 성능 개선을 확인하기 위해 실험을 진행하였다. 실험 환경으로는 MD의 개수, EN의 개수, 에피소드 개수 등을 설정하였다. EN은 공통으로 5개로 가정했고, MD는 30에서 150까지 20단위로 설정을 하였다. MD는 작업을 EN에 전체 할당, 또는 아예 할당하지 않는 이진 오프로드로 설정했다. 각 에피소드는 100개의 타임슬롯을 가지며, 각 길이는 0.1초로 설정했다. 학습률은 0.001로, 배치 크기는 32로 고정했으며, 옵티마이저는 RMSprop을 사용했다. 학습은 Google Colab pro에서 L4 GPU를 사용했으며, 시스템 RAM은 53GB, GPU RAM은 22.5GB, 디스크는 201.2GB 환경에서 실험을 진행했다.

5.1 MDs 개수에 따른 각 모델별 완료 태스크 수

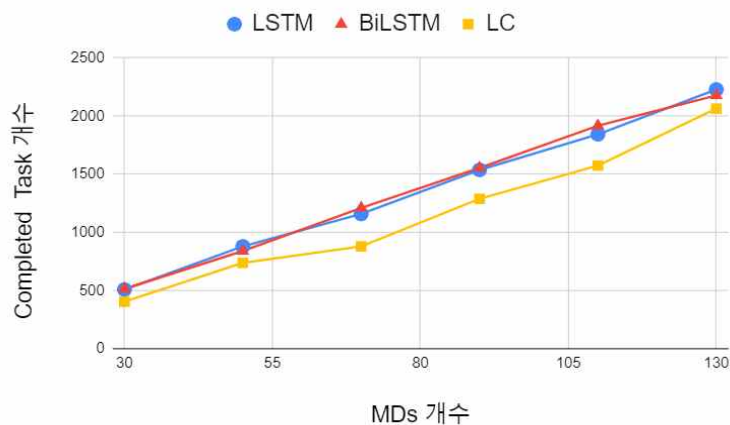


Fig. 2. Graph of the number of tasks completed for each model according to MDs

위 그래프는 MDs 개수에 따른 모델별 완료 태스크 개수를 나타낸다. BiLSTM과 LSTM을 적용한 모델이 모든 작업을 Local Computing(LC) 했을 때보다, 더 뛰어난 모습을 보인다. BiLSTM 적용 모델이 LC와 비교했을 때, 평균 잔차는 약 21%로 더 성능이 뛰어나다. BiLSTM과 LSTM 적용 모델을 비교했을 때는 약 1%의 차이로 BiLSTM이 더 유리한 모습을 보인다.

5.2 에포크 수에 따른 모델별 드랍율과 드랍 태스크 개수

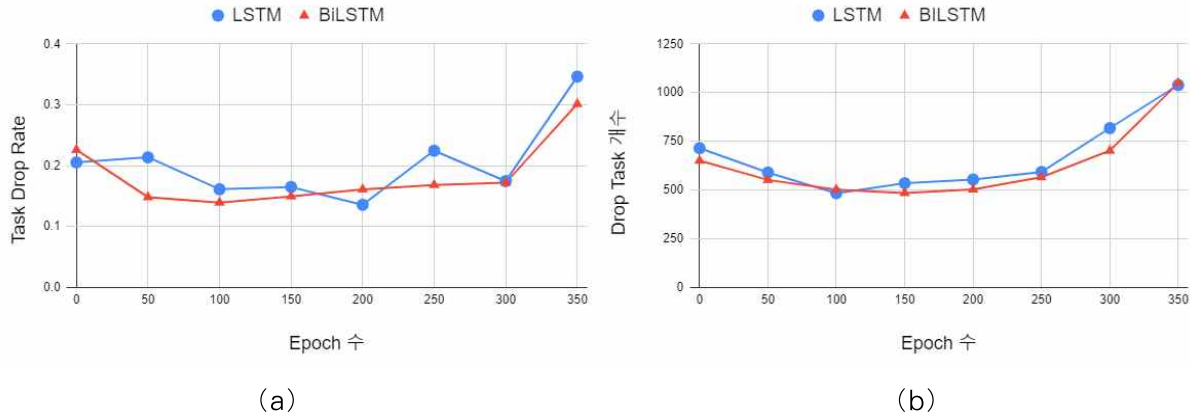


Fig. 3. Task drop rate and task by model according to epoch. (a) task drop rate (dropped task number /all task number), (b) drop task

위의 (a) 그래프는 에포크에 따른 모델별 드랍율이다. BiLSTM과 LSTM 적용 모델을 비교하였고, 에포크 200일 때를 제외하면, BiLSTM이 더 뛰어난 것을 보인다. 평균 잔차 비교했을 때, 약 8%로 BiLSTM이 LSTM보다 성능이 더 좋았다. (b) 그래프는 에포크에 따른 모델별 드랍 태스크 개수이다. 그래프를 보면 BiLSTM일 때, LSTM보다 에포크 100일 때를 제외하면, 전체적으로 근소하게 유리한 모습을 보인다. 평균 잔차 계산 시, 약 6% 차이로 BiLSTM이 더 뛰어나다.

5.3 제안한 모델을 사용하며 MDs 개수가 110일 때, 에포크에 따른 MD의 평균 딜레이, 에너지

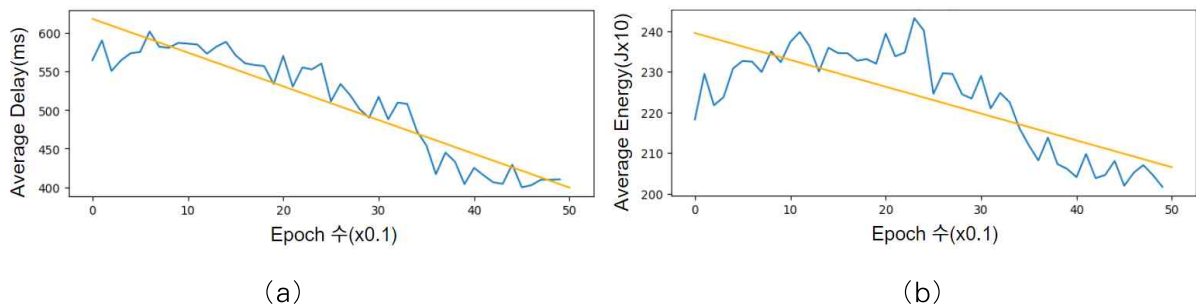


Fig. 4. When using the proposed model and the number of MDs is 110, the average delay and energy graph of MD according to epoch. (a) average delay, (b) average energy

위는 제안한 BiLSTM을 적용한 모델을 사용하고 MDs 개수가 110일 때, 에포크에 따른 MD의 평균 지연과 에너지를 나타내는 그래프이다. 밑의 x축은 에포크를 10으로 나눈 값으로, 예시로 에포크가 100이면 10으로 표시했다. 에포크가 10일 때 마다, MD의 평균 지연과 에너지를 나타낸다. 그래프를 보면, 학습을 진행할수록, 작업에 걸리는 평균 지연 및 에너지가 감소함을 볼 수 있다. 각 값에 대한 추세선의 식은 $y = -4.39x + 618.37$, $y = -0.66x + 239.51$ 으로 나타났으며, 결정계수 R^2 은 0.87, 0.59를 보여주었다. 모든 값은 소수점 둘째 자리까지 반올림하여 나타냈다.

6. 결 론

본 논문에서는 모바일 엣지 컴퓨팅을 위한 BiLSTM을 이용한 심층 강화 학습 기반 태스크 오프로딩 알고리즘을 제안하였다. 제안하는 기법은 기존 QOCO에서 사용한 LSTM을 BiLSTM으로 변경하여, 모델이 워크로드 변화를 잘 고려하게 되어 성능을 개선하였다. BiLSTM은 양방향 학습으로 시계열 예측에서 기존 모델보다 더 좋은 성능을 보였다. 기존 모델 대비 MDs의 완료한 태스크 개수는 평균 약 1%, 드랍율 및 드랍 태스크 개수는 약 8%, 6%의 성능 향상을 보였다.

REFERENCE

- [1] I. Rahmati, H. Shah-Mansouri, and A. Movaghar, "QOCO: A QoE-Oriented Computation Offloading Algorithm Based on Deep Reinforcement Learning for Mobile Edge Computing," arXiv preprint arXiv:2311.02525, 2023.
- [2] Ahn, J. and Lee, B. M., "IoT Roaming Service for Seamless IoT Service," *Journal of Korea Multimedia Society*, vol. 23, no. 10, pp. 1258-1269, 2020.
- [3] T.X. Tran and D. Pompili, "Joint Task Offloading and Resource Allocation for Multi-Server Mobile-Edge Computing Networks," *IEEE Transactions on Vehicular Technology*, Vol. 68, No. 1, pp. 856-868, 2018.
- [4] R. Liang, B. Yang, Z. Yu, X. Cao, D.W.K. Ng, and C. Yuen, "A Multi-Head Ensemble Multi-Task Learning Approach for Dynamical Computation Offloading," Proceedings of the 2023 IEEE Global Communications Conference, pp. 6079-6084, December 2023.
- [5] X. Wang, Y. Han, V.C. Leung, D. Niyato, X. Yan, and X. Chen, "Convergence of Edge Computing and Deep Learning: A Comprehensive Survey," *IEEE Communications Surveys and Tutorials*, Vol. 22, No. 2, pp. 869-904, 2020.
- [6] A.H.D. B. Deep, "Dynamic Computation Offloading with Energy Harvesting Devices: A Hybrid Decision Based Deep Reinforcement Learning Approach," 2020.
- [7] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas, "Dueling Network Architectures for Deep Reinforcement Learning," Proceedings of the International Conference on Machine Learning, pp. 1995-2003, June 2016.
- [8] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, Vol. 9, No. 8, pp. 1735-1780, 1997.
- [9] M. Schuster and K.K. Paliwal, "Bidirectional Recurrent Neural Networks," *IEEE Transactions on Signal Processing*, Vol. 45, No. 11, pp. 2673-2681, 1997.
- [10] P. Baldi, S. Brunak, P. Frasconi, G. Soda, and G. Pollastri, "Exploiting the Past and the Future in Protein Secondary Structure Prediction," *Bioinformatics*, Vol. 15, No. 11, pp. 937-946, 1999.
- [11] S. Siامي-Namini, N. Tavakoli, and A.S. Namin, "The Performance of LSTM and BiLSTM in Forecasting Time Series," Proceedings of the 2019 IEEE International Conference on Big Data, pp. 3285-3292, December 2019.

6. 저자약력

- 제1저자

- 現) 국민대학교 소프트웨어학부 (2020년3월 ~ 현재)
- UROP(2024년3월 ~ 6월), 하계방학 학부생 인턴쉽(2024년 8월 ~ 9월)
- 인공지능, 엣지 컴퓨팅, 웹/앱 프로그래밍



- 교신 저자

- 現) 국민대학교 소프트웨어학부 교수 (2019년9월 ~ 현재)
- 前) 삼성전자 Senior Professional (1998년1월 ~ 2017년3월)
- 네트워크, 통신, 클라우드, 엣지 컴퓨팅, 인공지능, 웹/앱 프로그래밍

